

Explanation - Arijeet Kundu

Reliable Data Transfer over UDP (CS-30003 CA1, P3)

Member: Arijeet Kundu (24052064) | Part: Stop-and-Wait, Checksums, Sequence Numbering

Your files: arijeet_kundu/checksum.py, arijeet_kundu/packet.py, arijeet_kundu/stop_and_wait.py

The 1-minute story (say this first in the viva)

- UDP is like posting letters without tracking: letters can get lost, arrive twice, arrive in the wrong order or arrive damaged. UDP itself does nothing about it.
- So **we** build reliability on top. My part is the basic toolkit that every protocol in our project uses: **packet format**, **sequence numbers** and **checksum**. On top of that I built the simplest protocol, **Stop-and-Wait**.
- **Checksum** = a number calculated from the data. If a byte changes in the network, the receiver calculates a different number and throws the packet away.
- **Sequence number** = a serial number on every packet (0, 1, 2 ...). It lets the receiver know the order and spot duplicates.
- **Stop-and-Wait** = send one packet, wait for the ACK (acknowledgement = 'got it'), then send the next. If no ACK arrives within the timeout, send the same packet again.

Packet layout (very important, draw this on paper in the exam)

[type : 1 byte] [sequence number : 4 bytes] [checksum : 2 bytes] [payload : up to 1024 bytes]

Header = 1 + 4 + 2 = **7 bytes**. type is DATA (1), ACK (2) or FIN (3, means 'last packet').

Stop-and-Wait picture

Sender: send #0 --> wait --> ACK 0 arrives --> send #1 --> wait ...

If #1 is lost: timer runs out --> send #1 again (this is a **retransmission**).

If ACK 1 is lost: sender resends #1 --> receiver sees an old sequence number (a duplicate) --> it does NOT store it again, only re-sends ACK 1.

Why it is slow: the sender is idle while waiting. Only one packet is in the network at a time. That is why our experiment shows Stop-and-Wait has the lowest goodput.

Words used in the code

- **chan** = our channel socket (a normal UDP socket wrapped by Shlok's emulator). We call chan.sendto(...) to send.
- **dest** = address (IP, port) we send to. **chunks** = the file cut into 1024-byte pieces.
- **timeout** = seconds to wait for an ACK. **retransmission** = sending the same packet again.
- **window / adaptive** are not used in Stop-and-Wait. They are only in the function so all three protocols have identical function signatures and main.py can call them the same way.

File: arijeet_kundu/checksum.py

Code	What it means (plain English)
<code>def calculate_checksum(data):</code>	Defines a function that takes bytes and returns one number (the checksum).
<code> total = 0 # running sum starts at zero</code>	Start a running total at 0.
<code> for byte in data: # go through every byte of the data total += byte # add the byte's value (0-255) to the sum</code>	Go through every byte in the data (each byte is a number 0-255) and add it to the total. So checksum = sum of all bytes.
<code> return total % 65536 # keep only 16 bits so it fits in 2 bytes</code>	% is remainder. $65536 = 2^{16}$, so we keep only the lowest 16 bits. That way the answer always fits in 2 bytes of the header.
<code>def verify_checksum(data, received_checksum):</code>	Function used by the receiver to check a packet.
<code> return calculate_checksum(data) == received_checksum</code>	Calculate the checksum again from the data we got and compare it with the checksum that was written in the packet. True = data is fine, False = corrupted.

File: arijeet_kundu/packet.py

Code	What it means (plain English)
<pre>import socket import struct import time</pre>	socket = for UDP errors (timeouts). struct = converts numbers to raw bytes and back. time = to measure how much waiting time is left.
<pre>from checksum import calculate_checksum, verify_checksum</pre>	Bring in my two checksum functions from checksum.py.
<pre>DATA = 1 # packet carries a piece of the file ACK = 2 # packet is an acknowledgement FIN = 3 # packet says "this was the last one"</pre>	Names for the three packet types, stored as numbers 1, 2, 3 in the header. Names are easier to read than magic numbers.
<pre>HEADER_FORMAT = "!BIH" # ! = network byte order, B = 1 byte, I = 4 bytes, H = 2 bytes</pre>	A recipe for struct. '!' = network byte order (standard for the internet). B = 1 byte (type), I = 4 bytes (sequence number), H = 2 bytes (checksum).
<pre>HEADER_SIZE = struct.calcsize(HEADER_FORMAT) # = 7</pre>	struct calculates the size of that recipe automatically: 1+4+2 = 7 bytes.
<pre>BUFFER_SIZE = 4096 # max bytes we read from the socket at once</pre>	The most bytes we read from the socket in one go. Our packets are ~1031 bytes so 4096 is plenty.
<pre>GIVE_UP_SECONDS = 20 # if nothing useful happens for this long, stop with an error</pre>	Safety net: if nothing useful happens for 20 seconds, stop with an error instead of hanging forever.
<pre>def make_packet(packet_type, seq, payload=b''):</pre>	Builds the raw bytes to send. payload defaults to empty (ACKs have no payload).
<pre> body = struct.pack("!BI", packet_type, seq) + payload</pre>	The part we protect with the checksum: type + sequence number + payload (the checksum field itself is not included, it doesn't exist yet).
<pre> checksum = calculate_checksum(body)</pre>	Compute the checksum of that body.
<pre> header = struct.pack(HEADER_FORMAT, packet_type, seq, checksum)</pre>	Now build the real 7-byte header with all three fields.
<pre> return header + payload</pre>	Glue header and payload together = the finished packet.
<pre>def parse_packet(raw):</pre>	Opposite of make_packet: takes received bytes and unpacks them. Returns None if the packet is bad.
<pre> if len(raw) < HEADER_SIZE: # too short to even hold a header return None</pre>	If fewer than 7 bytes arrived, it can't even contain a header. Reject.
<pre> packet_type, seq, checksum = struct.unpack(HEADER_FORMAT, raw[:HEADER_SIZE])</pre>	Take the first 7 bytes and split them into the three header fields.
<pre> payload = raw[HEADER_SIZE:]</pre>	Everything after the 7th byte is the payload (file data).
<pre> body = struct.pack("!BI", packet_type, seq) + payload</pre>	Rebuild exactly the same body the sender used for its checksum.
<pre> if not verify_checksum(body, checksum): # data was corrupted -> throw away return None</pre>	If our calculated checksum differs from the header's, some byte got corrupted, so we discard the packet (acts like it was lost, then retransmission fixes it).
<pre> return packet_type, seq, payload</pre>	Packet is good: hand back its three parts.

Code	What it means (plain English)
<code>def receive_packet(sock, timeout):</code>	Waits for ONE good packet for at most `timeout` seconds. All protocols use this helper.
<code> deadline = time.time() + timeout</code>	Compute the exact clock time when we must stop waiting.
<code> while True:</code>	Keep looping: bad packets are skipped and we try again until the deadline.
<code> remaining = deadline - time.time() # how much waiting time is left</code>	How many seconds are still left.
<code> if remaining <= 0:</code> <code> return None</code>	Time is over -> return None, which means 'timeout, nothing arrived'.
<code> sock.settimeout(remaining)</code>	Tell the socket: don't block longer than the remaining time.
<code> try:</code> <code> raw, _ = sock.recvfrom(BUFFER_SIZE)</code>	Try to read one datagram. <code>recvfrom</code> gives (bytes, sender address); we ignore the address (the <code>_</code>).
<code> except socket.timeout: # nothing arrived in time</code> <code> return None</code>	Socket waited the full time and nothing came -> timeout -> None.
<code> except ConnectionResetError: # Windows quirk, just ignore it</code> <code> continue</code>	On Windows, sending UDP to a closed port can raise this weird error. We just ignore it and keep waiting.
<code> parsed = parse_packet(raw)</code>	Check the packet (checksum etc.).
<code> if parsed is not None: # good packet -> give it back</code> <code> return parsed</code>	Good packet -> return it. If it was corrupted, the loop simply goes around again.

File: arijeet_kundu/stop_and_wait.py

Code	What it means (plain English)
<code>import time</code>	Needed to run timers.
<code>from packet import make_packet, receive_packet, DATA, ACK, FIN, GIVE_UP_SECONDS</code>	Use my packet tools from packet.py.
<code>def send(chan, dest, chunks, window, timeout, adaptive=True):</code>	The sender side. Same parameters as the other protocols; window and adaptive are ignored here.
<code> items = [(DATA, c) for c in chunks] + [(FIN, b'')] # file pieces + a final FIN</code>	Make a list of things to send: every file piece as a DATA packet, plus one final FIN packet (empty) that tells the receiver 'the file is finished'.
<code> packets_sent = 0 retransmissions = 0</code>	Two counters for our experiment results.
<code> for seq, (packet_type, payload) in enumerate(items): # seq = 0, 1, 2, ...</code>	Go through the items one by one. enumerate gives numbers 0,1,2,... which become the sequence numbers. So 'one packet at a time' is literally this for-loop.
<code> packet = make_packet(packet_type, seq, payload)</code>	Build the actual bytes for this packet.
<code> acked = False first_try = True started = time.time()</code>	acked = have we got the ACK yet? first_try = is this the first time sending? started = when we began this packet (for the give-up safety check).
<code> while not acked:</code>	Keep (re)sending this packet until its ACK arrives. This is the retransmission loop.
<code> chan.sendto(packet, dest) # (re)send the packet packets_sent += 1</code>	Send it through the channel emulator and count it.
<code> if not first_try: retransmissions += 1 # this was a repeat</code>	If this is not the first attempt, it is a retransmission - count it.
<code> first_try = False</code>	From now on any further send is a retransmission.
<code> deadline = time.time() + timeout # start the timer</code>	Start the timer: we will wait until this clock time.
<code> while not acked: remaining = deadline - time.time()</code>	Inner loop: wait for the ACK. remaining = time left on the timer.
<code> if remaining <= 0: break # timer expired -> resend</code>	Timer finished without our ACK -> leave the inner loop; the outer loop will resend.
<code> reply = receive_packet(chan, remaining)</code>	Wait for a good packet, but no longer than the remaining time.
<code> if reply is None: break # timer expired -> resend</code>	Nothing came in time -> timeout -> resend.
<code> reply_type, reply_seq, _ = reply</code>	Split the reply into type and sequence number (payload not needed).
<code> if reply_type == ACK and reply_seq == seq: acked = True # the ACK we wanted!</code>	We only accept an ACK that has OUR sequence number. Old/duplicate ACKs for earlier packets are ignored. When it matches, we move on to the next packet.

Code	What it means (plain English)
<pre> if time.time() - started > GIVE_UP_SECONDS: raise TimeoutError("stop-and-wait: receiver not responding")</pre>	If we have been trying one packet for 20 s, the receiver is probably dead. Stop with an error.
<pre> return {"packets_sent": packets_sent, "retransmissions": retransmissions}</pre>	Give the counters back to main.py for the report and plots.
<pre>def receive(chan, dest, window, linger):</pre>	The receiver side. dest = where to send ACKs. linger = how long to wait after the last packet.
<pre> expected = 0 # the sequence number we are waiting for</pre>	Sequence number we are waiting for next. Starts at 0.
<pre> chunks = [] # file pieces received so far, in order</pre>	List where we store the file pieces in order.
<pre> finished = False # becomes True once FIN has arrived</pre>	Becomes True when the FIN packet has arrived.
<pre> while True:</pre>	Receive forever until we decide to stop.
<pre> wait = linger if finished else GIVE_UP_SECONDS</pre>	Before FIN: be patient (20 s). After FIN: only wait 'linger' seconds more - because our last ACK might have been lost and the sender may resend FIN; we must be around to re-ACK it.
<pre> packet = receive_packet(chan, wait)</pre>	Get the next good packet (corrupted ones are already dropped).
<pre> if packet is None: if finished: return chunks # all done</pre>	Silence after FIN = everything is done. Return the file pieces.
<pre> raise TimeoutError("stop-and-wait: sender went silent")</pre>	Silence BEFORE FIN = something is wrong -> error.
<pre> packet_type, seq, payload = packet</pre>	Unpack the packet.
<pre> if packet_type == ACK: continue # not for us</pre>	The receiver only expects DATA/FIN; ignore ACKs.
<pre> if seq == expected: # the packet we were waiting for</pre>	Is this exactly the packet we wanted? (Right sequence number = new data, not a duplicate.)
<pre> if packet_type == FIN: finished = True else: chunks.append(payload)</pre>	If it is the FIN, remember we're finished. Otherwise it is file data: store it.
<pre> expected += 1</pre>	Now wait for the next sequence number.
<pre> if seq < expected: # new or duplicate -> ACK it chan.sendto(make_packet(ACK, seq), dest)</pre>	If seq is now below 'expected', it was either just accepted or an old duplicate. Either way send ACK for that sequence number. Re-ACKing duplicates is what fixes lost ACKs. (A packet with seq > expected can't happen in Stop-and-Wait.)

Viva questions you may be asked (with easy answers)

Q. Why do we need checksums if UDP already has one?

A. UDP's checksum is optional and only protects the UDP layer; the assignment asks us to build our own. Ours is used by all three protocols and lets our emulator's corruption be detected.

Q. What kind of checksum is yours? Is it perfect?

A. It is the sum of all bytes modulo 65536. It always detects a single changed byte. It is not perfect - two changes that cancel each other out can slip through (real systems use CRC), but for our project it is simple and enough.

Q. Why sequence numbers in Stop-and-Wait if only one packet is in flight?

A. Because ACKs and packets can be duplicated or delayed. Without numbers the sender could accept an old ACK as the ACK of the current packet, and the receiver could store a duplicate twice. (Textbooks use 0/1 alternating; we use continuous numbers which is simpler.)

Q. What happens if the DATA packet is lost? If the ACK is lost?

A. DATA lost: no ACK, timer expires, sender resends. ACK lost: sender resends, receiver sees an old sequence number, does not store it again, just sends the ACK again.

Q. Why does the receiver 'linger' after FIN?

A. The two generals problem: the sender can't know if its FIN arrived except through the ACK. If that last ACK is lost the sender resends FIN; a receiver that already quit would never answer. Lingering 1 second lets it re-ACK.

Q. Why is Stop-and-Wait slow?

A. Only one packet per round-trip time. Throughput is about packet size / RTT no matter how fast the link is. Our plots show it as the lowest line.

Q. What is the timeout and how did you choose it?

A. How long to wait for an ACK. We use 0.1 s: bigger than our emulated round-trip (about 20-30 ms) but not so big that losses waste lots of time.

Q. What does `struct.pack('!BIH', ...)` do?

A. It converts the numbers into raw bytes: 1 byte, 4 bytes, 2 bytes, in network byte order. `struct.unpack` does the reverse.

Q. What is a retransmission and where do you count it?

A. Sending a packet again. In `stop_and_wait.send`, every send after the first for the same sequence number increments the retransmissions counter.